

Bases de datos

Miguel García Moreno

2025-12-30

Contents

Ejercicio 1. Diagrama entidad-relación	3
Ejercicio 2. Consultas SQL	4
Ejercicio 3. PyMySQL	5
Ejercicio 4. MongoDB	7
Pregunta 1	7
Pregunta 2	7
Pregunta 3	8
Pregunta 4	8
Ejercicio 5. MongoEngine	8

Ejercicio 1. Diagrama entidad-relación

Se desea realizar la base de datos para la gestión de un banco. Para ello se deberá almacenar la información de los clientes (DNI, nombre, domicilio y teléfono) y de las cuentas bancarias (número de cuenta, fecha de apertura y tipo: cuenta corriente, cuenta nómina, cuenta ahorro, o cuenta joven). Cada cliente puede ser titular de múltiples cuentas y cada cuenta puede tener múltiples titulares (todo titular de una cuenta debe ser cliente del banco; a efectos de la base de datos, un cliente y un titular representan la misma entidad). Adicionalmente, se desea almacenar la información de las tarjetas emitidas por el banco. Cada tarjeta tiene una numeración y un tipo (crédito o débito), y está asociada a un único titular y a una única cuenta bancaria.

Además, para la gestión bancaria se desean almacenar dos tipos de operaciones en la base de datos: - Transferencias. Una transferencia se realiza entre dos cuentas bancarias (una de origen y otra de destino), y tiene un importe y una fecha. - Operaciones con tarjetas. Cada operación con tarjeta (como compras) tiene un importe y una fecha, y está asociada a una única tarjeta.

Realice el diagrama entidad-relación para representar adecuadamente dicha información. Para cada entidad, especifique todos sus atributos y su clave primaria (PK), y para cada relación su cardinalidad y sus atributos, si los hubiera.

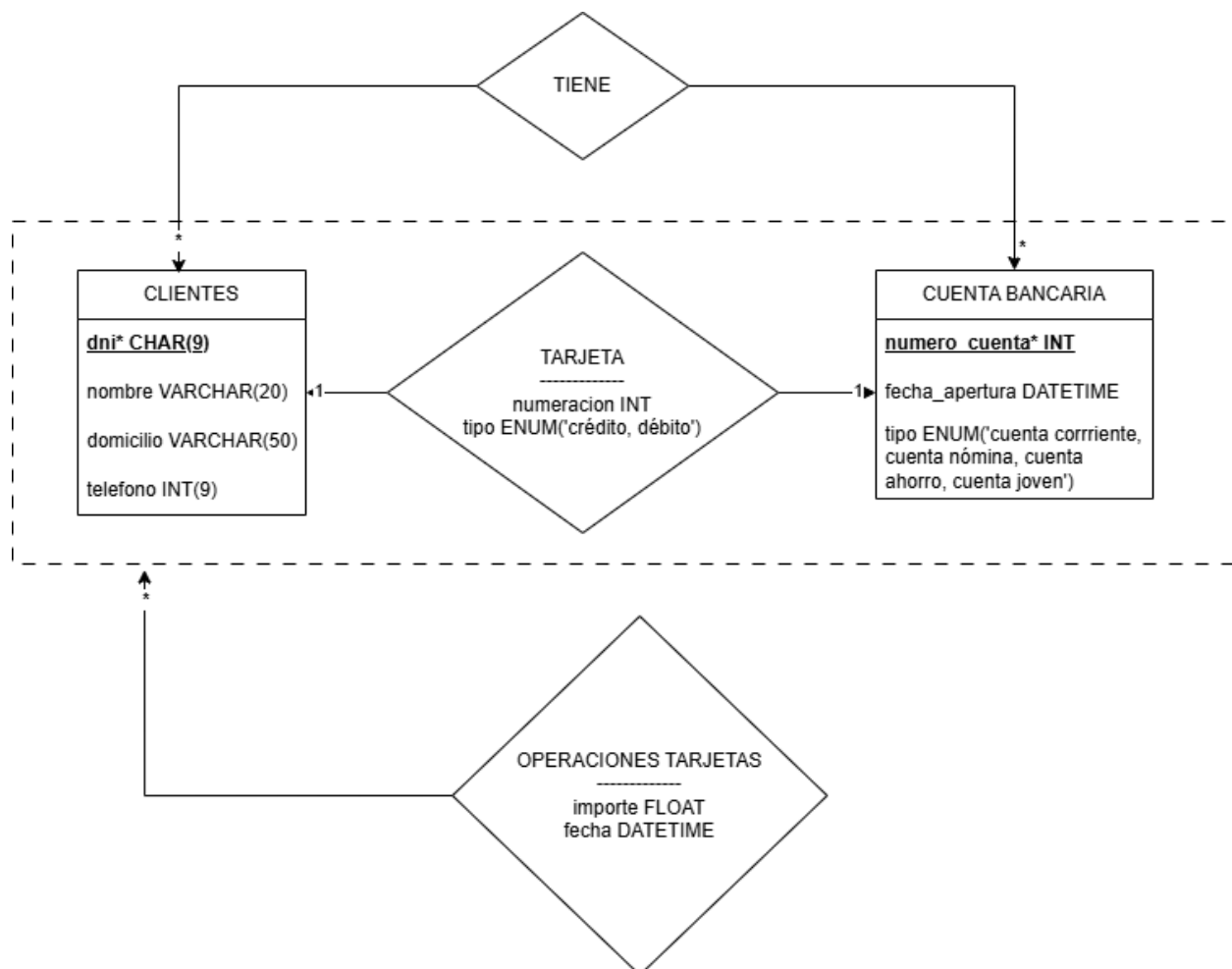


Figure 1: MODELO_ER

Ejercicio 2. Consultas SQL

A continuación se muestra el diseño lógico (estructura de tablas) de una base de datos dedicada a la gestión de un videoclub.

CLIENTE (dni^{PK} , nombre, dirección, teléfono)

PELÍCULA (id^{PK} , nombre, año)

ACTOR (id^{PK} , nombre, fechaNac, nacionalidad)

REPARTO ($idPelicula^{PK,FK}$, $idActor^{PK,FK}$)

ALQUILER ($dniCliente^{PK,FK}$, $idPelicula^{PK,FK}$, Fecha)

Se pide el código SQL, respetando nombres de tablas y de atributos, que realice las siguientes consultas:

- Consultar todas las películas existentes en el videoclub, mostrando únicamente la información de las películas (id, nombre y año).

```
/*Primera consulta: Listar todas las películas (id, nombre y año) almacenadas en la base de datos. */
SELECT P.id AS ID_Pelicula,
       P.nombre AS Nombre_Pelicula,
       P.año AS Año_Pelicula,
FROM PELÍCULA P
```

- Consultar todos los actores que participen en más de una película, mostrando únicamente la información de los actores (id, nombre, fechaNac y nacionalidad).

```
/*Segunda consulta: Listar los actores (nombre, fecha de nacimiento, id y nacionalidad) que han participado en más de una película, junto con el número de películas en las que han participado. */
-- PRIMERA OPCION, SIN USAR SUBCONSULTA
SELECT  A.nombre AS Nombre_Actor,
        A.fechaNac AS Fecha_Nacimiento,
        A.id AS ID_Actor,
        A.nacionalidad AS Nacionalidad_Actor
FROM ACTOR A
LEFT JOIN REPARTO R
  ON A.id = R.idActor
GROUP BY A.id, A.nombre, A.fechaNac, A.nacionalidad
HAVING COUNT(R.idPelicula) > 1

-- SEGUNDA OPCION, USANDO SUBCONSULTA
SELECT  Nombre_Actor,
        Fecha_Nacimiento,
        ID_Actor,
        Nacionalidad_Actor
FROM (
  SELECT  A.nombre AS Nombre_Actor,
          A.fechaNac AS Fecha_Nacimiento,
          A.id AS ID_Actor,
          A.nacionalidad AS Nacionalidad_Actor,
          COUNT(R.idPelicula) AS Num_Peliculas
  FROM ACTOR A
```

```

LEFT JOIN REPARTO R
  ON A.id = R.idActor
GROUP BY A.id, A.nombre, A.fechaNac, A.nacionalidad ) AS Subconsulta
WHERE Num_Peliculas > 1;

```

- Consultar todos los clientes que han alquilado alguna película en la que participa algún actor que ha participado en más de una película, mostrando únicamente la información de los clientes (DNI, nombre, dirección y teléfono).

```

/*Tercera consulta: Consultar todos los clientes que han alquilado alguna
película en la que participa algún actor que ha participado en más de una
película, mostrando únicamente la información de los clientes
(DNI, nombre, dirección y teléfono) */
SELECT C.dni AS DNI_Cliente,
       C.nombre AS Nombre_Cliente,
       C.dirección AS Dirección_Cliente,
       C.teléfono AS Teléfono_Cliente
FROM CLIENTE C,
/*Tabla artificial con los actores que han participado en más de una película.
Se podría usar tabla temporal por mejorar tiempos de ejecucion. */
(SELECT ID_Actor
FROM (
  SELECT A.id AS ID_Actor,
         COUNT(R.idPelicula) AS Num_Peliculas
  FROM ACTOR A
  LEFT JOIN REPARTO R
    ON A.id = R.idActor
  GROUP BY A.id ) AS Subconsulta
WHERE Num_Peliculas > 1) AS Actores_Multiples_Peliculas
-- Filtro de clientes que han alquilado peliculas con actores de la tabla "actores_multiples_peliculas"
WHERE C.dni IN (
  SELECT A.dniCliente
  FROM ALQUILER A, REPARTO R
  WHERE A.idPelicula = R.idPelicula
        AND R.idActor IN (SELECT ID_Actor FROM Actores_Multiples_Peliculas) );
--Esta misma consulta se podria hacer con JOINS.

```

Ejercicio 3. PyMySQL

Realice un script Python basado en PyMySQL que cree la base de datos relacional necesaria para almacenar el contenido del siguiente fichero CSV, y que procese los datos de dicho fichero para insertarlos en la BD. Nótese que, como todo fichero CSV, las columnas están separadas por comas, y que las líneas que comienzan por el carácter '#' son líneas de comentarios. Además, la tercera columna (teléfono) es un string que contiene una lista de números de teléfonos separados por el carácter punto y coma ';'. Use la estructura de base de datos que considere adecuada para almacenar estos datos.

nombre, apellido, teléfonos

Carmen,García,654213900

Manuel,Jiménez,666111234;608523478;612333456

Carmen,González,634555555;619033412

Nota: el código debe procesar cualquier fichero de entrada tipo CSV que siga el formato anterior y que tenga un número indeterminado de líneas. El ejercicio NO consiste en meter a mano las tres líneas anteriores

DDL script para la base de datos en mysql:

```
create database clients
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

use clients;

create table client (
    `id` int auto_increment primary key,
    `name` varchar(20) not null,
    `surname` varchar(40) not null,
    unique(`name`,`surname`)
) ;

create table telephone_number (
    client_id int not null,
    tlf_number int primary key,
    constraint fk_client foreign key (client_id) REFERENCES client(id)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);
```

Python script para automatización de inserción de registros en base de datos:

```
conexion = obtener_conexion_bd("localhost", "root", "n>0${6S31P3i}", "clients")

def insertar_datos(conexion, csv : str):
    """
    Inserta datos de un archivo CSV en la tabla 'clientes' de la base de datos.
    EL archivo CSV debe tener el formato:
    # nombre, apellido, teléfonos
    Carmen,García,654213900
    Manuel,Jiménez,666111234;608523478;612333456
    Carmen,González,634555555;619033412
    Args:
        conexion (pymysql.connections.Connection): Objeto de conexión a la base
        de datos.
        csv (str): Ruta al archivo CSV con los datos a insertar.
    """
    cursor = conexion.cursor()

    with open(csv, 'r', encoding='utf-8') as file:
        next(file) # Saltar la cabecera
        for line in file:
            nombre, apellido, telefonos = line.strip().split(',')
            telefonos_list = telefonos.split(';')

            cursor.execute(
                "INSERT INTO client (name, surname) VALUES (%s, %s)",
                (nombre, apellido)
```

```

    )
    client_id = cursor.lastrowid
    conexion.commit()

    for telefono in telefonos_list:
        cursor.execute(
            """INSERT INTO telephone_number (client_id, tlf_number) VALUES
            (%s, %s)""",
            (client_id, telefono)
        )
        conexion.commit()
    cursor.close()

insertar_datos(conexion, 'prueba_tlf.csv')

```

Ejercicio 4. MongoDB

Responda razonadamente a las siguientes preguntas:

Pregunta 1

Ponga un ejemplo práctico concreto de una base de datos para la cuál sería más conveniente usar un modelo no-relacional como el que implementa MongoDB antes que un modelo relacional como el que implementa MySQL. Justifique su respuesta.

Es el caso de una base de datos para una red social pues una red social tiene la necesidad de registrar datos muy heterogéneos. Esta flexibilidad para el escalamiento horizontal es una característica de las bases de datos no relacionales. Además, el constante cambio en la recogida de la información de estas redes sociales, o bien en las características de las mismas aplicaciones hacen que de nuevo esta sea la mejor forma para escalar sus bases de datos. Pongamos por ejemplo que esta red social implementa una nueva característica en la que permite subir un tipo de video de un formato determinado. Esta información se tendrá que recoger en la base de datos para posteriormente mostrarse a los usuarios en sus respectivos feeds. Probablemente esta nueva característica sea más conveniente guardarla en una colección o documento existente dentro de la propia base de datos no relacional, y sería tan sencillo como directamente añadirlo desde las sentencias de inserción que produce el backend, sin necesidad de modificar la propia estructura de base de datos y por supuesto evitando las validaciones que ocurren en los modelos relaciones. Finalmente, si esta red social hubiera usado un modelo relacional debería haber creado una estructura, a saber, mediante la creación de una nueva entidad y relaciones, o modificación del modelo de datos, para poder soportar la nueva funcionalidad y almacenar los datos.

Pregunta 2

Explique las ventajas y desventajas de MySQL y MongoDB en la inserción de datos y en la ejecución de consultas. Justifique su respuesta.

MySQL tiene la capacidad de mantener la consistencia de los datos por la construcción relacional de su modelo de datos. Esto impide redundancias o incoherencias en el modelo de datos. La inserción de datos es, por tanto, una transacción segura e informativa, porque en el caso de entrar un intento de registro en conflicto con algunas de las restricciones en tabla se produce un ROLLBACK de la transacción, evitando errores, y además se emite un mensaje que indica qué restricción (FK, unique, ...) es la que causa el

conflicto. Dado que estas validaciones tienen que tener lugar, MySQL tiene un mayor tiempo de inserción que MongoDB, porque esta segunda al no hacer verificaciones es muy rápida.

Por otro lado, en el terreno de las consultas a datos ya almacenados en la base de datos, MySQL es mucho más eficiente y además permite realizar consultas muy complejas. Al tener una estructura rígida de datos MySQL es capaz de acceder más ágilmente a ellos y además incorpora funciones nativas con las que construir consultas tan complejas como se requieran. Ejemplos de estas funciones nativas son: JOIN, RANK, PARTITION BY, LIKE, HAVING, GROUP BY, ... En MongoDB la consulta de datos consume más tiempo, ya que el motor no tiene una información estructurada de antemano. Esto implica que los algoritmos de consulta debe buscar sin conocer cómo van a ser los documentos, derivando en una mayor demora en la consulta.

Pregunta 3

Especifique el filtro para mostrar todos los documentos de una colección en MongoDB que o bien tienen el campo “anchura” con valor mayor que 100, o bien tienen el campo “altura” con valor menor que 50.

```
db.coleccion.find({
  $or: [
    { anchura: { $gt: 100 } },
    { altura: { $lt: 50 } }
  ]
})
```

Pregunta 4

Especifique el filtro para mostrar todos los documentos de una colección que contengan un array cuyo nombre sea “keywords” y que dicho array contenga la palabra clave “mongodb”.

```
db.coleccion.find({
  keywords: "mongodb"
})
```

Ejercicio 5. MongoEngine

Realice un script Python basado en MongoEngine que procese y almacene en una base de datos MongoDB un fichero JSON similar al que se muestra a continuación. Use la estructura de objetos y documentos que considere adecuada para ello.

```
{nombre:Carmen,apellido:García,tfn:654213900}
{nombre:Manuel,apellido:Jiménez,tfn:666111234; 608523478;612333456}
{nombre:Carmen,apellido:González,email:carmen@ema.il}
```

Nota 1: en el fichero anterior, sólo existen los campos “nombre”, “apellido”, “tfn” e “email”, siendo los dos primeros obligatorios, y los dos últimos opcionales. Además, se permite que cada persona tenga un número indefinido de teléfonos y de emails asociados.

Nota 2: al igual que en el Ejercicio 3, el código debe procesar cualquier fichero de entrada tipo JSON que siga el formato anterior y que tenga un número indeterminado de líneas. El ejercicio NO consiste en meter a mano los tres documentos anteriores.


```

%pip install mongoengine
from mongoengine import *

connect('mydatabase', host='localhost', port=27017)

class User(Document):
    nombre = StringField(required=True)
    apellido = StringField(required=True)
    tlfn = ListField(StringField())
    email = ListField(StringField())

def create_user(nombre, apellido, tlfn_list = None, email=None):
    """
    Crea un usuario con la información proporcionada.

    Args:
        nombre (str): Nombre del usuario.
        apellido (str): Apellido del usuario.
        tlfn_list (list of str, optional): Lista de números de teléfono del
        usuario. Defaults to None.
        email (list of str, optional): Lista de correos electrónicos del usuario.
        Defaults to None.
    """
    user = User(
        nombre=nombre,
        apellido=apellido,
        tlfn=tlfn_list,
        email=email
    )
    user.save()

def save_users_from_txt(file_path):
    """
    Lee un archivo de texto con información de usuarios y los guarda en la base
    de datos.
    El formato del archivo es una especie de pseudojson y debe ser:
    {nombre:John, apellido:Doe, tfno:123456789;987654321, email:email1@email.com;
    email2@email.com}
    {nombre:Jane, apellido:Smith, email:email3@email.com}
    {nombre:Alice, apellido:Johnson, tfno:555555555}

    Este formato que no es JSON complica el parseo manual del archivo, y fuerza a
    implementar una lógica específica para extraer los datos.

    Args:
        file_path (str): Ruta al archivo de texto con la información de los usuarios.
    """
    with open(file_path, 'r', encoding='utf-8') as file:
        for line in file:
            # Limpiar la línea y eliminar llaves
            line = line.strip().strip('{}')

            data = {}

```

```

# Parsear la línea manualmente en un diccionario
for part in line.split(','):
    key, value = part.split(':', 1)
    data[key.strip()] = value.strip()

# Extraer los campos necesarios
nombre = data['nombre']
apellido = data['apellido']

telefonos = []
emails = []

# Extraer teléfonos y emails si existen
if 'tfno' in data:
    telefonos = data['tfno'].split(';')

if 'email' in data:
    emails = data['email'].split(';')

# crear el usuario en la base de datos
create_user(nombre, apellido, telefonos, emails)

# Ejemplo de uso
save_users_from_txt('prueba.txt')

```